

A Full Delivery:

1. The robot will first move to the `pickup_waypoint`
2. Requests a `DispenserRequest` till receives a `DispenserResult` . (Done Dispensing)
3. Continue delivery and moves to `dropoff_waypoint`
4. Requests a `IngestorRequest` till receives a `IngestorResult` . (Done Ingesting)

Found a bug? [Edit this page on GitHub](#).

RMF Web

Introduction

Open-RMF Web provides a comprehensive web application toolkit, with an API server, re-usable frontend components, as well as a customizable frontend dashboard.

For more information on setup and documentation, please refer to the `rmf-web` repo.

Open-RMF

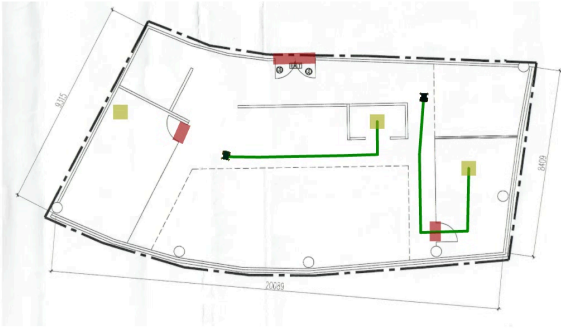
MAPSYSTEM OVERVIEWTASKS

+ NEW TASK

Map

Levels

L1



OSRC-SG

Open-RMF

MAPSYSTEM OVERVIEWTASKS

+ NEW TASK

Robots

Name	Fleet	Est. Task Finish Time	Level	Battery	Last Updated	Status
tinyRobot1	tinyRobot	-	L1	100.00%	1/1/1970, 7:34:28 AM	IDLE
tinyRobot2	tinyRobot	1/1/1970, 7:35:11 AM	L1	100.00%	1/1/1970, 7:34:28 AM	WORKING

1-2 of 2

Doors

Name	Op. Mode	Current Floor	Type	Door State	
coe_door	ONLINE	L1	Single Swing	CLOSED	OPEN CLOSE
hardware_door	ONLINE	L1	Single Swing	CLOSED	OPEN CLOSE
main_door	ONLINE	L1	Double Swing	CLOSED	OPEN CLOSE

1-3 of 3

Lifts

Name	Op. Mode	Current Floor	Destination Floor	Lift State
No lifts available.				

0-0 of 0

Beacons

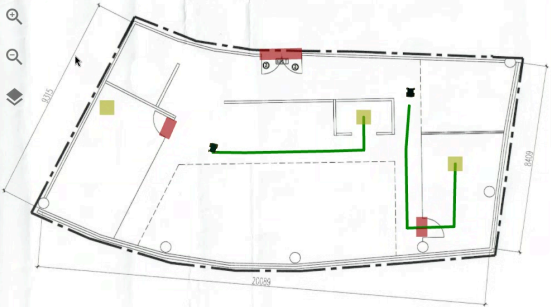
Name	Op. Mode	Level	Type	Beacon State
No beacons available.				

0-0 of 0

Map

Levels

L1



OSRC-SG

Mutex Groups

Group	Locked	Waiting
No rows		

0-0 of 0

open-RMF

MAP

SYSTEM OVERVIEW

TASKS

+ NEW TASK

🔔

⚠️

👤

Tasks

EXPORT PAST 31 DAYS

REFRESH TASK QUEUE

QUEUE

SCHEDULE

Date	Requester	Pickup	Destination	Robot	Start Time	End Time	State
31 Jul 2024	stuf	n/a	pantry	tinyRobot1	7:37:06 AM	7:37:37 AM	underway (state)
31 Jul 2024	stuf	n/a	hardware_2	tinyRobot2	7:37:01 AM	7:37:51 AM	underway (state)
31 Jul 2024	stuf	n/a	pantry	tinyRobot1	7:34:30 AM	7:35:04 AM	completed
31 Jul 2024	stuf	n/a	hardware_2	tinyRobot2	7:34:25 AM	7:35:16 AM	completed

1-1 of 1 < >

Map

Levels

L1

🔍

🔍

📍

OSRC-96

For users looking to set up a full deployment of Open-RMF with its core libraries, building infrastructure adapters, fleet adapters and Open-RMF web, please refer to the [Open-RMF deployment template](#).

Related Repositories

- [rmf-web](#)
- [Open-RMF deployment template](#)

Found a bug? [Edit this page on GitHub.](#)

UI

Certain portion of this doc is deprecated, please refer to [rmf-web](#) for the latest development.

Introduction

This chapter describes how we can write end user applications which integrate into RMF. We will briefly go through the requirements of an RMF UI application and provide a tutorial to write a React Webapp that interfaces with RMF.

Overview

The most common way to write a UI application is to make use of a UI framework. There are many available UI frameworks, the following are some of the popular frameworks for reference:

- Multi-Paradigm
 - Qt
 - React native
 - Xamarin
 - Flutter
- Web
 - React
 - Angular
 - Vue
- Desktop
 - wxWidgets
 - Gtk
 - WPF (Windows only)

Each framework has pros and cons and we do not feel there is a "best" framework for all use cases. That being said, some frameworks are more suitable to integrate with RMF due to their specific underlying technologies. To understand why, let's take a brief overview at how an RMF UI application would work.

How Does an RMF UI Application Work?

Imagine a simple UI application with a label that shows the current status of a door and a button that opens/closes the door. Recall that in the door adapter, we publish door states and subscribe to door requests. Here we are doing the reverse; subscribing to door states and publishing door requests. We will be listening to door states coming in from RMF and updating our display to match, and we will also be sending door requests when the user clicks the open or close button.

We need to use a library to help us do that. In the hardware adapter examples we are using the `rclcpp` library, but `rclcpp` isn't the only way to use ROS 2. Here are some libraries that also work well

- Direct
 - `rcl` (C)
 - `rclcpp` (C++)
 - `rclpy` (python)
 - `rclnodejs` (nodejs)
- Indirect
 - `SOSS` (websocket)
 - `rosbridge` (websocket)

A "direct" library is able to send and receive ROS 2 messages directly from the application, in contrast, an indirect library uses a middleman service to forward ROS 2 messages. In general, the "direct" approach is more preferable but sometimes it is not possible on the target platform. In these cases, the indirect approach can be used.

Note: This is not an exhaustive list, you can find many third-party projects that aim to bring the ROS 2 ecosystem to more platforms, the `ros2-dotnet` project for example, provides C# bindings for ROS 2. You can also write your own bindings and middlewares!

Writing an RMF UI application isn't much different from writing any other UI application, the only difference being that we will be sending/receiving ROS 2 data in response to user input and updating the UI's state.

Tutorial: React WebApp

In this section, we will go through an example of creating a React-based webapp to monitor door states and send door open/close requests. The tutorial will focus on the aspects of communicating with RMF; basic knowledge of React and TypeScript is needed.

Note that this isn't the only way to create a RMF UI application, as mentioned before, you can use any UI toolkit, the only requirement is to be able to send/receive ROS 2 messages.

The code for this tutorial is available [here](#).

Requirements

- nodejs >= 10
- [rmf_core](#)
- [soss](#)
- [rmf-soss-ros2](#)
- [rmf_demos](#)

We will not go through the process of setting up the dependencies, instructions to set them up can be easily found on the web or from their project homepages.

Setting Up

We will be using an example from `rmf_demo` as the RMF deployment that we will be interacting with. This is a good time to test if your installation of `rmf_demo` is working properly, start the demo with the following command:

```
ros2 launch demos office.launch.xml
```

Next let's test if SOSS is working. In order to run SOSS, you need to provide it with a configuration file; you can use [this](#) template as a start. You would need a certificate for SOSS; refer to various tutorials online to generate one. If you are using a self-signed cert, also make sure your browser is set to accept it for websocket connections. Add the path of your cert and key to the SOSS config and try starting SOSS with:

```
soss <path_to_config>
```

Once the environment is set up, we can then proceed to set up our actual app. First, create a React workspace with:

```
npx create-react-app react-app-tutorial --template typescript
```

Go inside the newly created `react-app-tutorial` directory and run the following command to install all the JavaScript dependencies we will be using:

```
npm install @osrf/romi-js-core-interfaces @osrf/romi-js-soss-transport  
jsonwebtoken @types/jsonwebtoken
```

These libraries are not strictly required but they contain helpful functions to use `soss` and to communicate with RMF. If you are building a JavaScript-based RMF app, it is recommended to make use of them, we will see later how they simplify the communication to RMF.

Note: Other than `@osrf/romi-js-soss-transport`, there is also `@osrf/romi-js-rclnodejs-transport` which is able to send ROS 2 messages directly, however it does not work on the browser. It is preferred if you are writing a Node.js based desktop application using something like Electron, or you are writing a server based application like a REST API provider.

Door Component

First of all, let's start things simple by creating a component to show a door's state along with open and close buttons. Create a new file called `Door.tsx` in `react-app-tutorial/src` directory and copy the below snippet into it.

```
import React from 'react';

export interface DoorProps {
  name: string;
  state: string;
  onOpenClick?(e: React.MouseEvent): void;
  onCloseClick?(e: React.MouseEvent): void;
}

export const Door = (props: DoorProps) => {
  const { name, state, onOpenClick, onCloseClick } = props;
  return (
    <div>
      Door: {name}
      <br />
      State: {state}
      <br />
      <button onClick={(e) => onOpenClick && onOpenClick(e)}>Open</button>
      <button onClick={(e) => onCloseClick && onCloseClick(e)}>Close</button>
      <br />
      <br />
    </div>
  );
};

export default Door;
```

Nothing much is happening here yet. We are simply rendering the door name, its state and open and a close button.

Let's test things out by running it. Replace `App.tsx` with this:

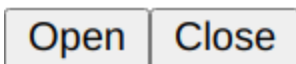
```
import React from 'react';
import Door from './Door';

function App() {
  return <Door name="example_door" state="Closed" />;
}

export default App;
```

and start it with `npm start`. You should see something like this:

Door: example_door
State: Closed



Great! We now have a base to start implementing the rest of the app.

Obtain List of Doors

Previously we made a simple door component and tested rendering it with hard-coded values. Obviously this won't work in a proper app, so here we will look at how we can obtain the list of actual doors from RMF.

Start of by adding a React state to track the list of doors:

```
const [doors, setDoors] = React.useState<RomiCore.Door[]>([]);
```

RMF has a `get_building_map` service that we can use to get the list of doors, lifts, levels among many other data, in order to make use of the service we need to do a ROS 2 service call, since there isn't ROS 2 support on the browser, we will be using an "indirect" approach with `sooss`. A websocket connection is made to SOSS which will then act as a middleman and pass our messages to the ROS 2 network.

An easy way to use SOSS is with the `@osrf/romi-js-soss-transport` package, so let's do it now. Add a react effect hook to your `App` component

```
React.useEffect(() => {
  (async () => {
    const token = jwt.sign({ user: 'example-user' }, 'rmf', { algorithm: 'HS256'
  });
    const transport = await SossTransport.connect('example',
'wss://localhost:50001', token);
  })();
}, []);
```

We will need to import `SossTransport`, so add this to the top of the file:

```
import { SossTransport } from '@osrf/romi-js-soss-transport';
```

This performs a websocket connection to the SOSS server. `example` is the ROS 2 node name that we will be using, and we are connecting to a SOSS server at `wss://localhost:50001`. The server uses a JWT token signed with a secret specified in the SOSS config. The example config is using `rmf`. If you changed the secret, be sure to change it here as well.

Note: This example is only for convenience, you should never reveal the secret to the client. Usually the client would connect to an authentication server which will verify that it is a valid request and return a signed token.

Now that we have a connection to SOSS, we can call the `get_building_map` service. Add this to the React effect:

```
const buildingMap = (await transport.call(RomiCore.getBuildingMap,
{})).building_map;
setDoors(buildingMap.levels.flatMap((level) => level.doors));
```

It uses `RomiCore`, so add this to your imports:

```
import * as RomiCore from '@osrf/romi-js-core-interfaces';
```

This downloads and parses the building map from RMF. `romi-js` simplifies a ROS 2 service call with the async `call` method. If you are familiar with `rclnodejs`, this is roughly equivalent to:

```
const client = node.createClient(
  'building_map_msgs/srv/GetBuildingMap',
  'get_building_map'
);
client.sendRequest({}, response => {
  const buildingMap = response.building_map;
  setDoors(buildingMap.levels.flatMap((level) => level.doors));
});
```

Notice that we need to provide it with type of the message (`building_map_msgs/srv/GetBuildingMap`) and the name of the service (`get_building_map`), but how do we find out what the service name and type are? We could read the RMF manual or query the ROS 2 system while RMF is running. Another way is with the help of `RomiCore`; it provides a list of known RMF services and messages so you don't have to go through the trouble of finding them yourself.

In the below statement, we are using `RomiCore` to call the `get_building_map` service without needing to know the service name and types:

```
transport.call(RomiCore.getBuildingMap, {})
```

Now that we have a list of `RomiCore.Door`, let's make things simpler by updating `Door.tsx` to take that in as a prop. While we're at it, let's also have it take in a `RomiCore.DoorState` as a prop since we will be using it later.

Door.tsx: